

Complex Computational Problem (CCP) — Artificial Neural Network
Neural Computing System for Handwritten Digit Recognition (MNIST)

Students: Ali Hamza (B23F0063AI106) & Zarmeena Jawad (B23F0115AI125)

Section: B.S AI Red

Instructor: Dr. Abid Ali

Department: IT and Computer Science

Institute: Pak-Austria Fachhochschule: Institute of Applied Sciences & Technology

Submission Date: 25-04-2026

Neural Computing System for Handwritten Digit Recognition (MNIST)

Ali Hamza & Zarmeena Jawad

Department of IT and Computer Science, Pak-Austria Fachhochschule

Abstract

This project implements a comprehensive neural computing system for recognizing handwritten digits (0–9) from the MNIST dataset using NumPy-only model implementations (external libraries are used strictly for MNIST loading). Four paradigms are implemented sequentially: (i) a single-layer Perceptron for baseline binary classification (0 vs 1) with decision boundary visualization, (ii) an Adaline network trained with the Delta Rule via gradient descent and analyzed for learning-rate convergence, (iii) a Kohonen Self-Organizing Map (SOM) for unsupervised topology-preserving clustering and anomaly detection via quantization error, and (iv) a fully connected backpropagation MLP ($784 \rightarrow 128 \rightarrow 64 \rightarrow 10$) for 10-class digit classification with activation and regularization comparisons plus SOM-feature integration. The MLP achieves 97.14% test accuracy on MNIST ($N = 10,000$) with a Wilson 95% confidence interval of 96.79%–97.45%. The SOM achieves quantization error $QE = 5.535$ and topographic error $TE = 0.1078$, enabling anomaly detection at a 95th-percentile threshold with precision = 0.8299, recall ≈ 1.0 , and $F1 = 0.9070$. Overall, the project bridges classical neural learning theory with modern implementation practice and compares each paradigm's strengths and limitations.

Index Terms

Perceptron, Adaline, Delta Rule, Kohonen SOM, Backpropagation, MLP, MNIST, NumPy-from-scratch, anomaly detection, topology preservation.

Neural Computing System for Handwritten Digit Recognition (MNIST)

Table of Contents

1. Course Learning Outcomes
 2. Project Overview
 3. I. Introduction
 4. II. Part A: Perceptron Model
 5. III. Part B: Delta Rule and Adaline
 6. IV. Part C: Kohonen SOM
 7. V. Part D: Backpropagation MLP
 8. VI. Comparative Analysis
 9. VII. Conclusion
 10. References
 11. Appendix A: Hyperparameters
-

List of Tables

Table I. Key results (summary)

Table II. Paradigm comparison

Table III. Hyperparameters (notebook run)

List of Figures

Fig. 1. Biological mapping to perceptron components

Fig. 2. Perceptron misclassifications per epoch

Fig. 3. Decision boundary (epoch 1)

Fig. 4. Decision boundary (epoch 2)

Fig. 5. Decision boundary (epoch 3)

Fig. 6. Decision boundary (epoch 6)

Fig. 7. Decision boundary (epoch 11)

Fig. 8. Decision boundary (epoch 35)

Fig. 9. Perceptron convergence (linearly separable)

Fig. 10. XOR failure (non-linearly separable)

Fig. 11. Adaline MSE surface

Fig. 12. Adaline gradient descent trajectory

Fig. 13. Adaline MSE vs epoch

Fig. 14. Adaline learning rate comparison

Fig. 15. SOM U-matrix

Fig. 16. SOM prototype (weight) maps

Fig. 17. SOM class hit map (winner)

Fig. 18. SOM hit strength map

Fig. 19. SOM anomaly QE histogram

Fig. 20. MLP loss curves

Fig. 21. MLP accuracy curves

Fig. 22. Confusion matrix

Fig. 23. Activation curves and gradients

Fig. 24. Activation comparison (val acc)

Fig. 25. Regularization effect (loss)

Fig. 26. SOM integration (val acc)

Neural Computing System for Handwritten Digit Recognition (MNIST)

Course Learning Outcomes

By the end of this project, students will be able to:

- Understand the biological basis of neural computation and implement a Perceptron model for binary classification tasks.
 - Apply the Delta Rule to train Adaline networks through gradient descent and analyze learning rate effects on convergence.
 - Design and train Kohonen Self-Organizing Maps for unsupervised feature learning, clustering, and topology-preserving dimensionality reduction.
 - Implement a multi-layer Backpropagation network from scratch to solve complex non-linear classification problems.
 - Evaluate and compare neural network models using performance metrics including accuracy, confusion matrices, and loss curves.
 - Integrate multiple neural learning paradigms into a unified system and critically analyze their strengths and limitations.
-

Project Overview

This project focuses on a comprehensive Neural Computing System capable of recognizing and classifying handwritten digit patterns (0–9) from the MNIST dataset. We implement four fundamental neural learning paradigms sequentially, each contributing a distinct role in the overall intelligent system pipeline. The project bridges classical neural computing theory with modern implementation practice.

Objectives

- Implement a single-layer Perceptron to establish a baseline binary classifier distinguishing digit classes.
 - Train an Adaline unit using the Delta Rule (Widrow–Hoff learning) and analyze gradient-based weight convergence.
 - Build and train a Kohonen Self-Organizing Map (SOM) to cluster digit feature representations in an unsupervised manner.
 - Design and implement a multi-layer Backpropagation network for full 10-class digit classification.
 - Evaluate each model with performance metrics and produce a comparative analysis of all four paradigms.
-

I. Introduction

Dataset

We use the MNIST handwritten digits dataset as the primary data source across all parts. MNIST consists of 28×28 grayscale images of digits 0–9. Each image is flattened to a 784-dimensional vector. Data is scaled to $[0, 1]$ and standardized where necessary for stable optimization in Part D.

Dataset sources: (1) <http://yann.lecun.com/exdb/mnist/>, (2) <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>, and (3) direct loading via `torchvision.datasets.MNIST` or `tensorflow.keras.datasets.mnist` (loading only).

Reproducibility

Notebook: `Assignments/Complex Computational Problem/notebooks/ccp_ann_project.ipynb`. Source implementations (NumPy only): `src/`. Figures: `figures/`. External frameworks are used only to load MNIST, not to implement models.

Evaluation Metrics

$$\text{Accuracy} = (1/N) \sum 1[y = \hat{y}]$$

$$\text{QE} = (1/N) \sum \|x - w_{\text{BMU}(x)}\|$$

$$\text{TE} = (1/N) \sum 1[\text{BMU}_1 \text{ and } \text{BMU}_2 \text{ are non-adjacent}]$$

Table I. Key Results (Summary)

Component	Metric	Value	Notes
Perceptron (0 vs 1, PCA-2D)	Test accuracy	0.99338	Binary subset for visual boundary evolution.
SOM	QE / TE	5.535 / 0.1078	Representative subset; topology-preserving clustering metrics.
SOM anomaly detection	Threshold + P/R/F1	7.4817; 0.8299/~1.0/0.9070	Threshold = 95th percentile of clean QE distribution.
MLP	Test accuracy	0.9714	MNIST test set ($N=10,000$); Wilson 95% CI: 0.9679–0.9745.

Note: The Perceptron boundary is plotted in PCA-2D for interpretability (not equivalent to full 784-D linear separability).

II. Part A: Perceptron Model

A(i) Biological Mapping and Historical Significance

The Perceptron is inspired by a biological neuron: dendrites correspond to input features, synapses to weights, the soma to a weighted summation with bias, and the axon to the output firing decision. Historically, Rosenblatt's Perceptron introduced a learnable linear classifier but is limited to linearly separable functions; it fails on XOR, motivating multi-layer networks.

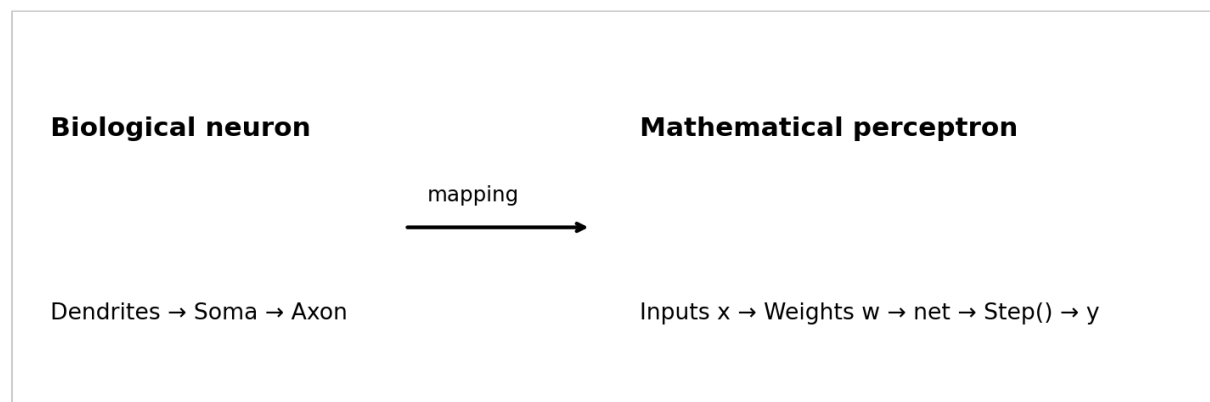


Fig. 1. Biological neuron mapping to Perceptron components (inputs, weights, summation, threshold output).

A(ii) Perceptron Implementation (NumPy Only)

The Perceptron uses a step activation and updates weights using:

$$w(t+1) = w(t) + \eta \cdot (d - y) \cdot x$$

Listing 1. Perceptron update loop (src/perceptron.py).

```
for _ in range(self.epochs):
    miscls = 0
    for xi, di in zip(xb, y):
        yi = 1 if (xi @ self.w) >= 0 else 0
        err = di - yi
        if err != 0:
            miscls += 1
            self.w = self.w + self.lr * err * xi
    self.history_["miscls"].append(int(miscls))
    self.history_["w"].append(self.w.copy())
```

A(ii) Decision Boundary Evolution (MNIST 0 vs 1 in PCA-2D)

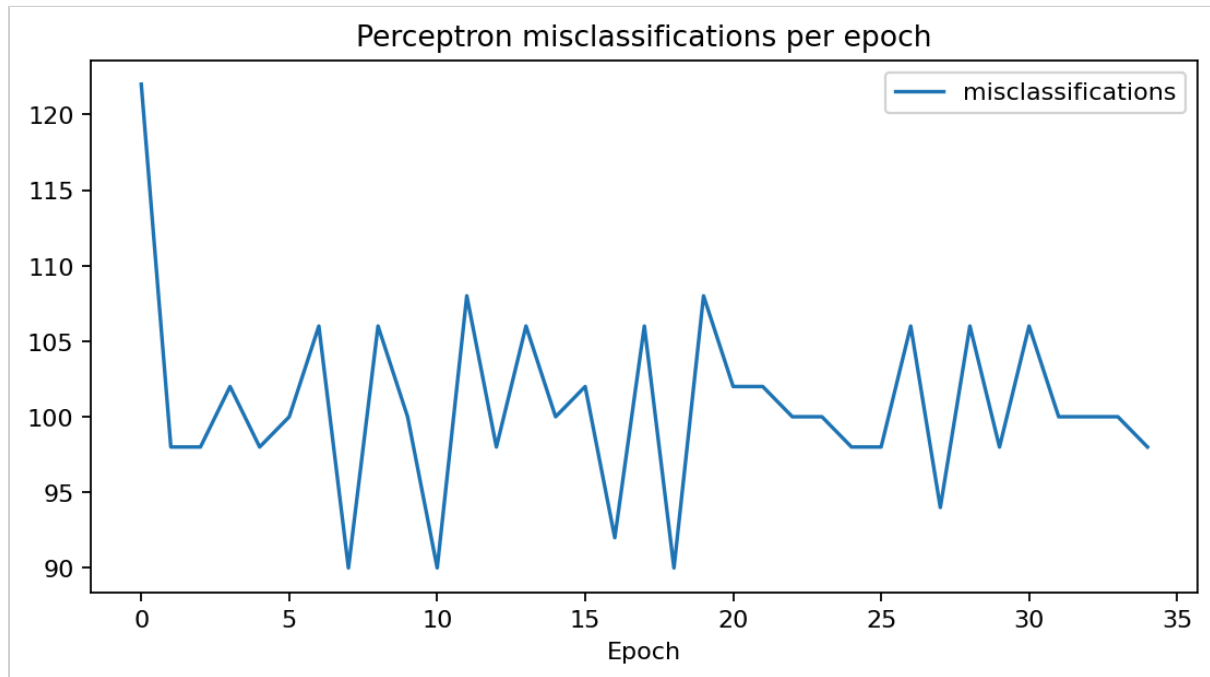


Fig. 2. Misclassifications per epoch for MNIST 0 vs 1 (PCA-2D).

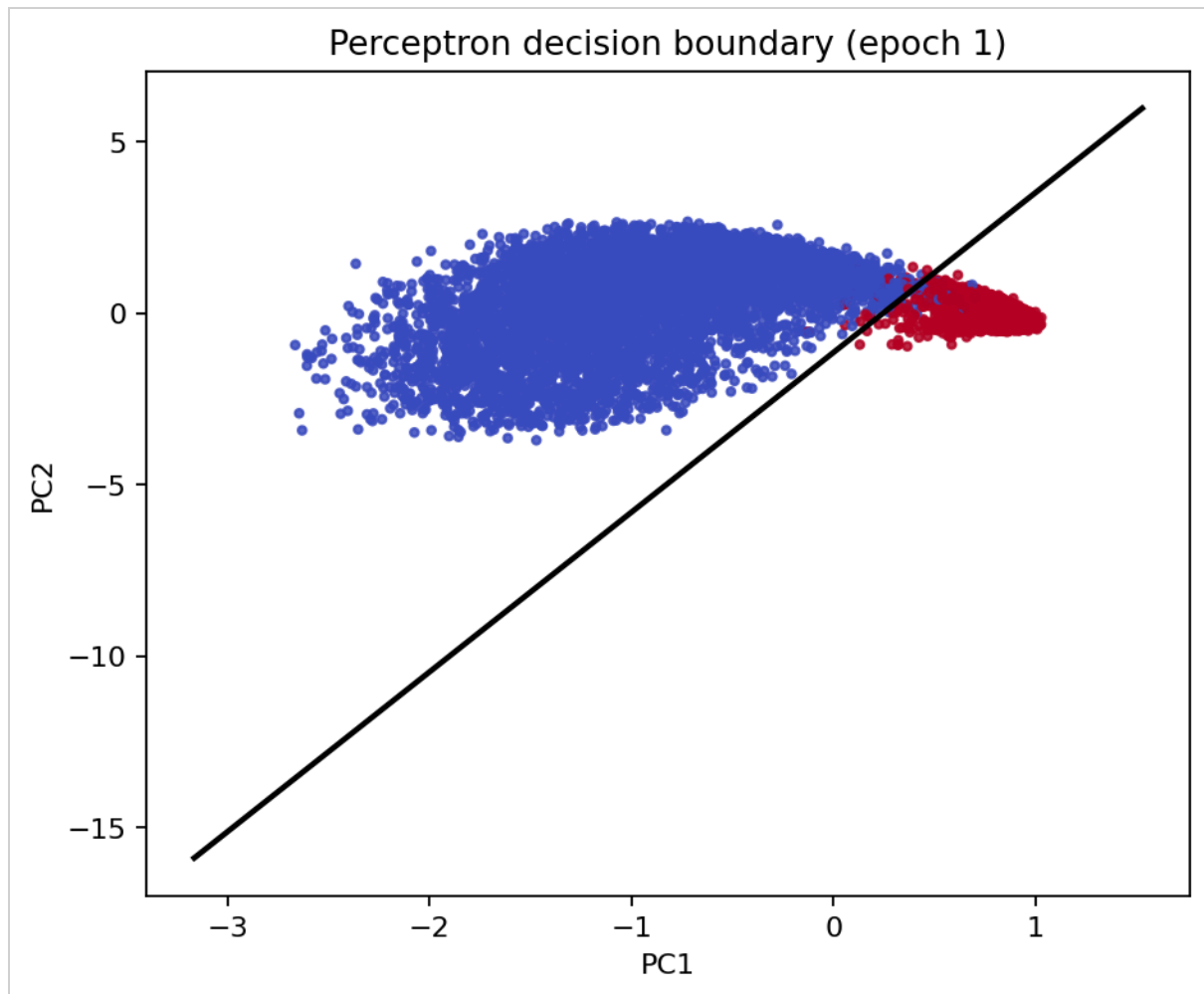


Fig. 3. Decision boundary at epoch 1 (PCA-2D).

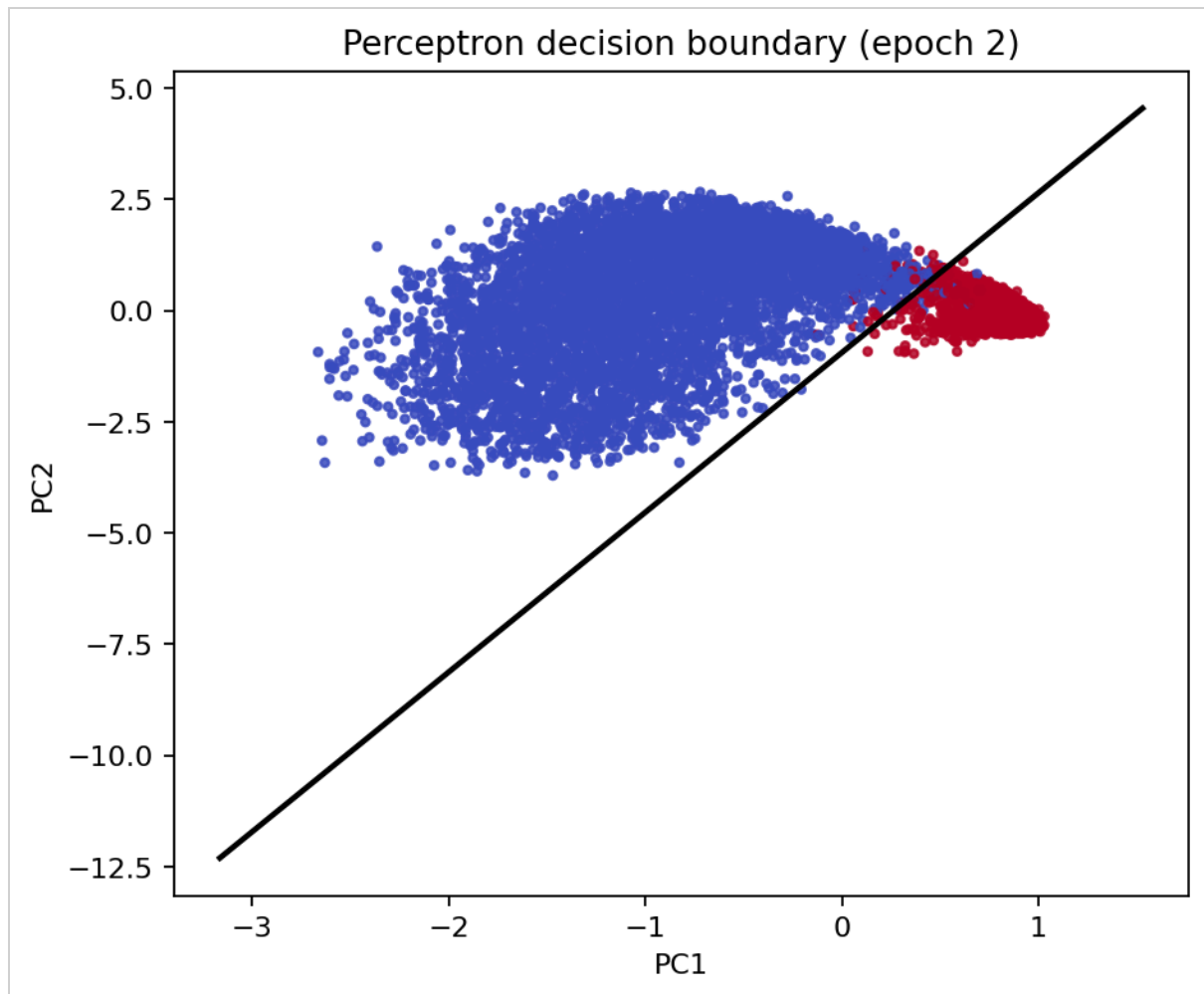


Fig. 4. Decision boundary at epoch 2 (PCA-2D).

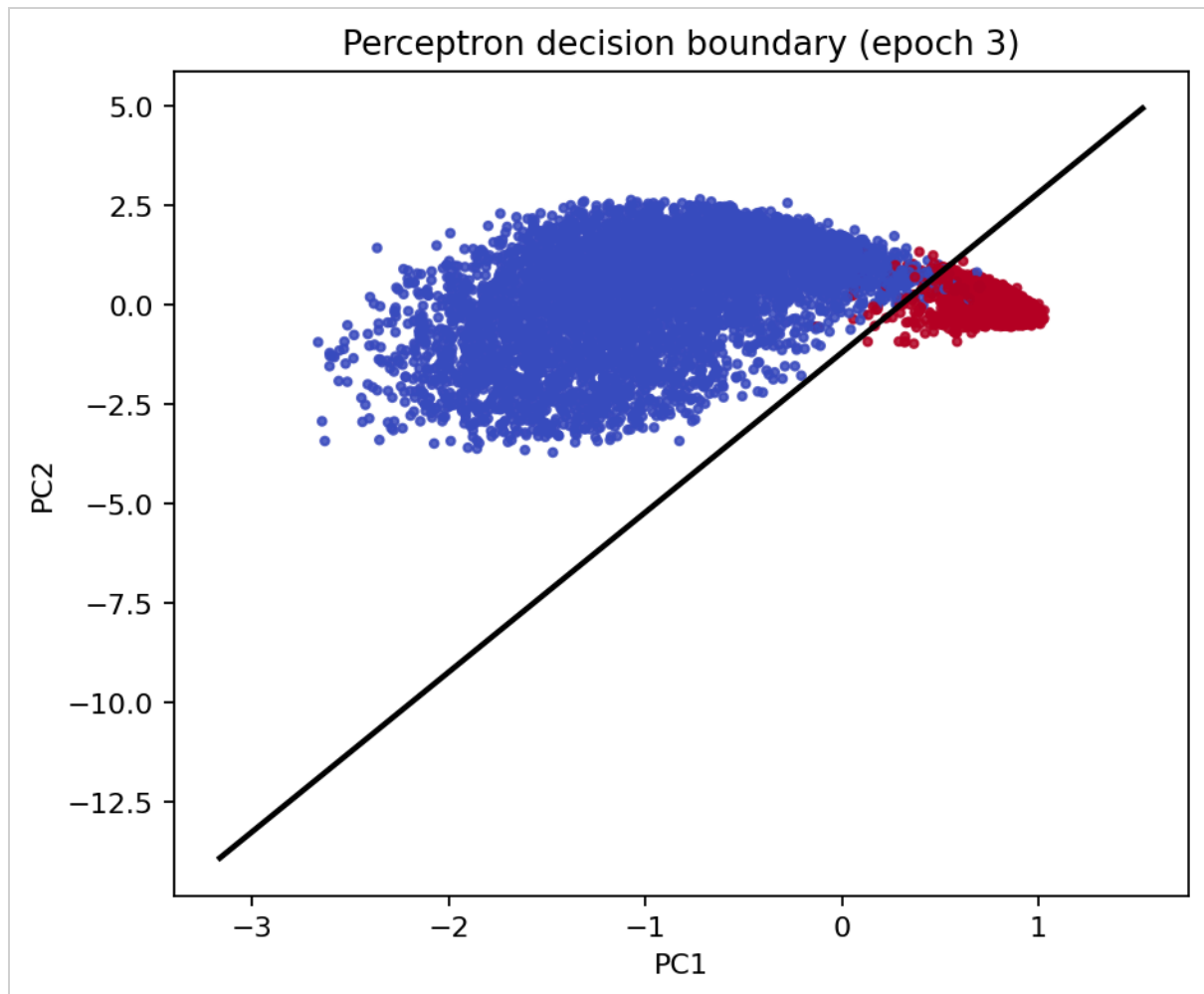


Fig. 5. Decision boundary at epoch 3 (PCA-2D).

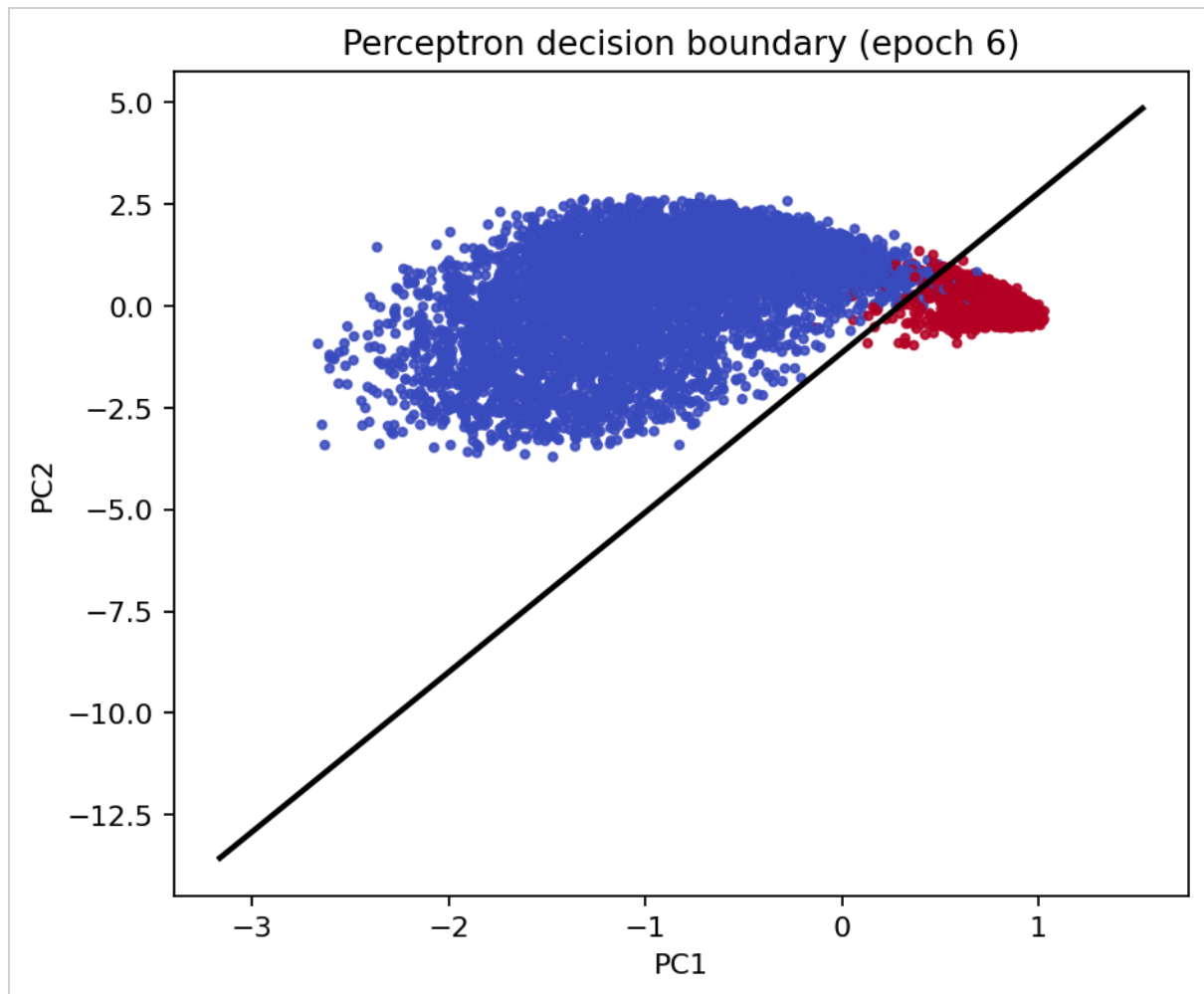


Fig. 6. Decision boundary at epoch 6 (PCA-2D).

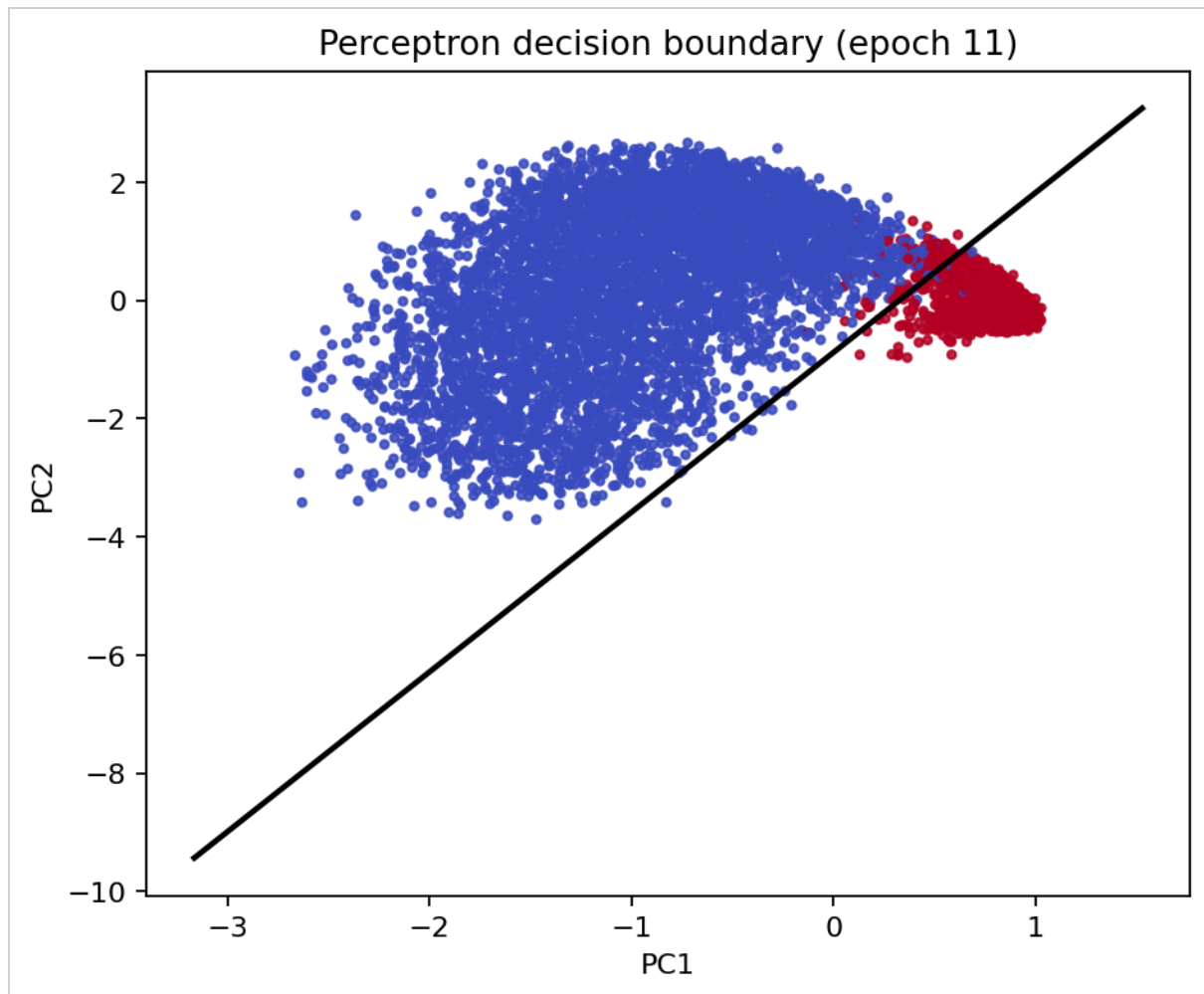


Fig. 7. Decision boundary at epoch 11 (PCA-2D).

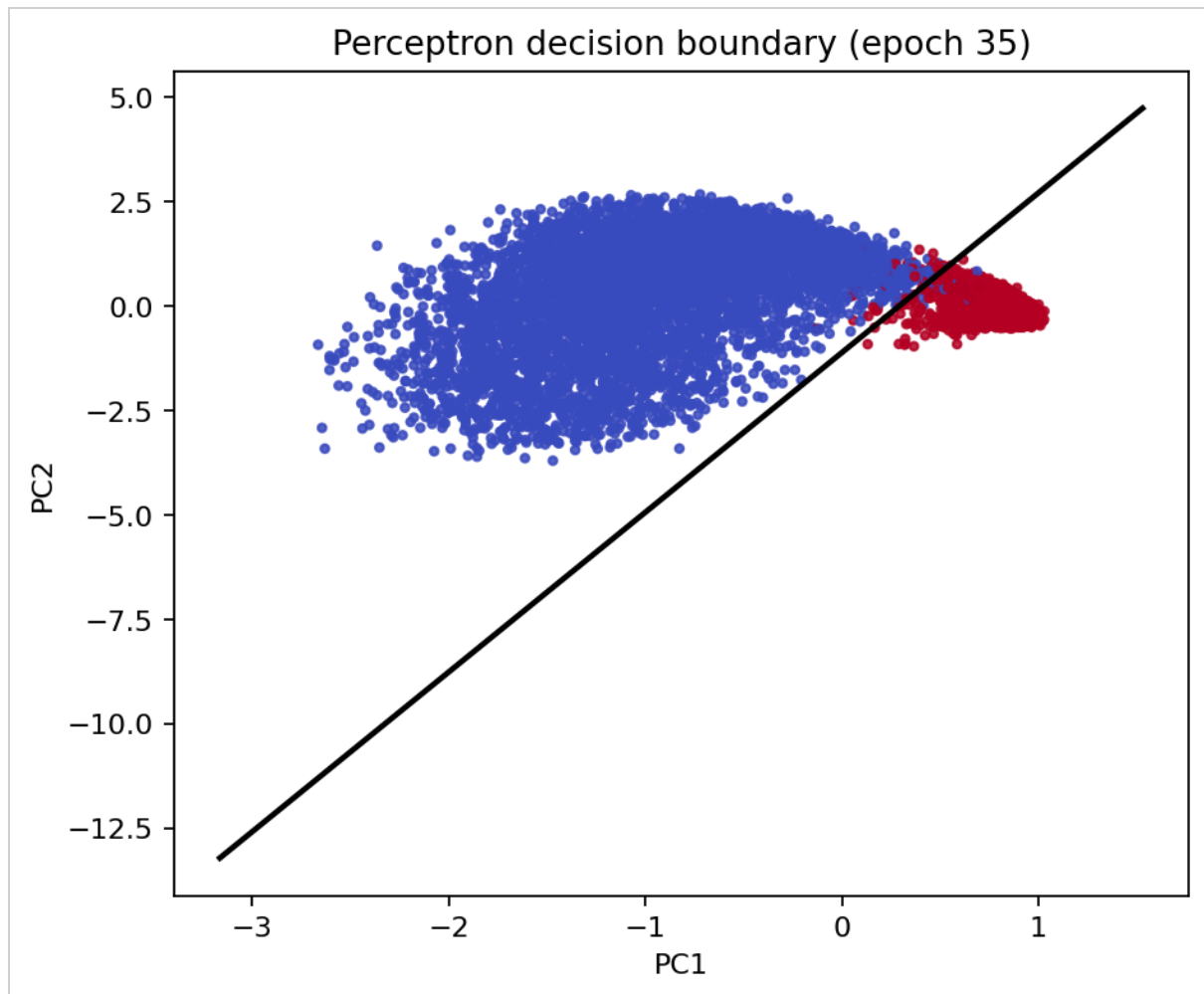


Fig. 8. Decision boundary at epoch 35 (PCA-2D). Final binary test accuracy: 0.99338.

A(iii) Convergence Analysis (Separable vs Non-Separable)

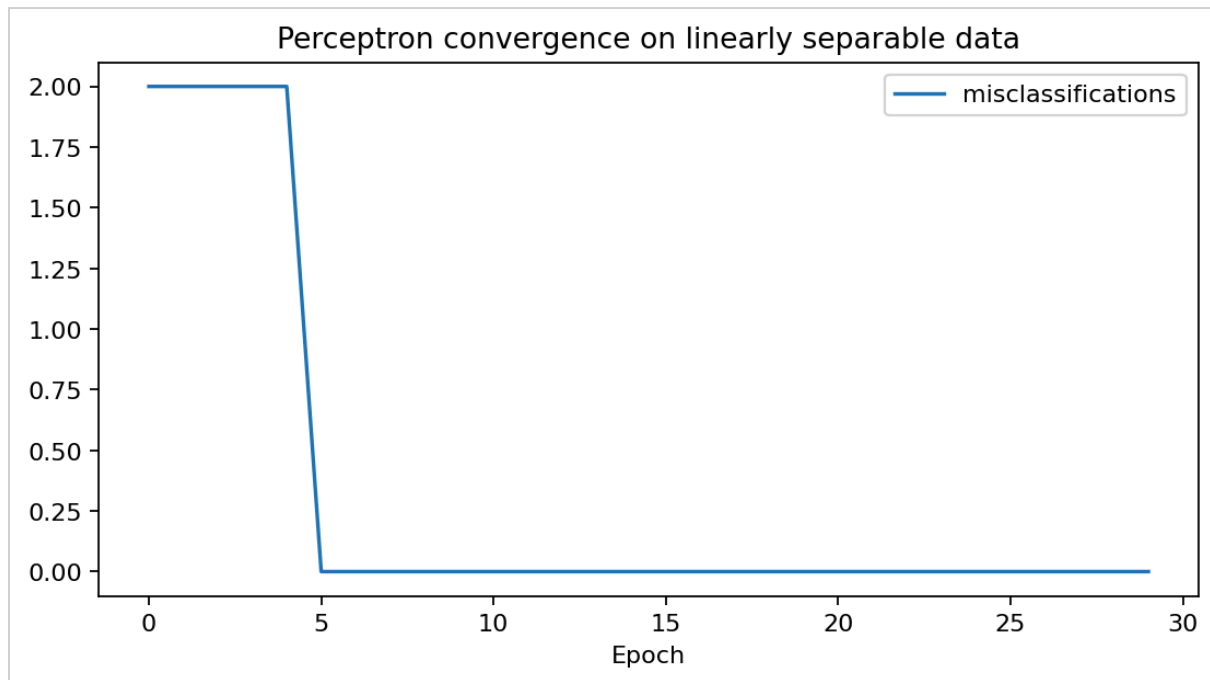


Fig. 9. Convergence behavior on linearly separable synthetic data.

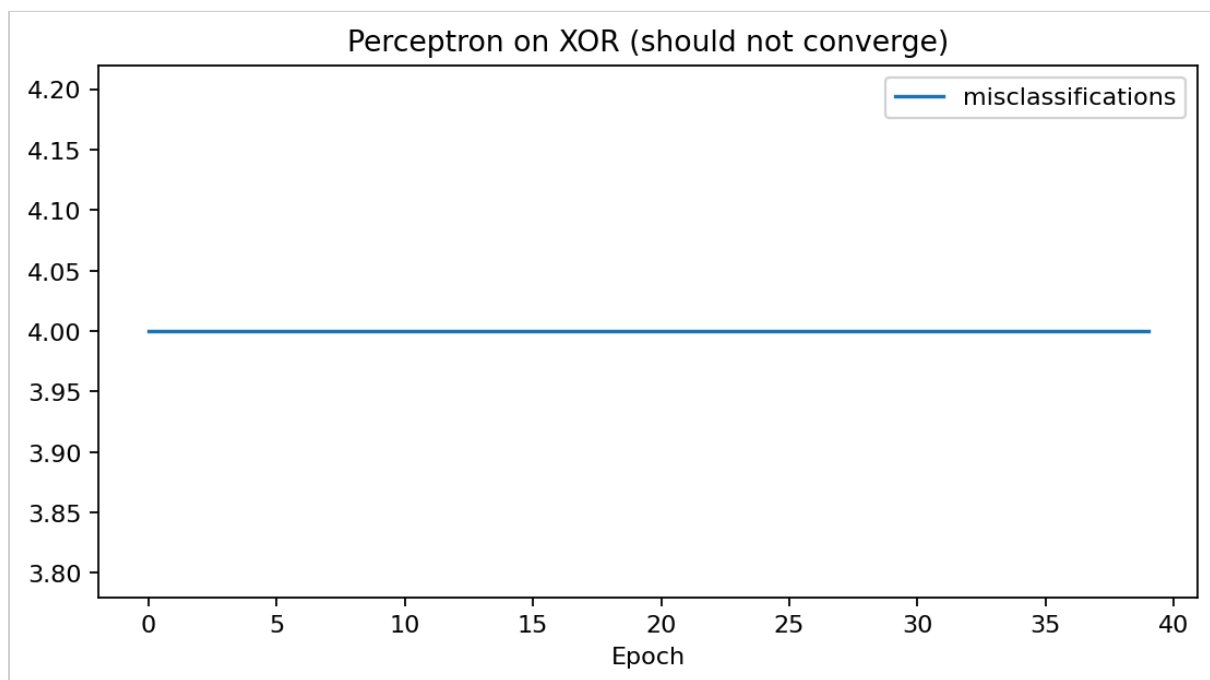


Fig. 10. XOR failure: no single hyperplane separates the classes; Perceptron does not converge to zero error.

III. Part B: Delta Rule and Adaline

B(i) Mathematical Derivation (Full Gradient Idea)

Adaline uses a continuous linear output during training and minimizes mean squared error (MSE) via gradient descent. Starting from $E = (1/2) \sum (d - \hat{y})^2$, differentiating with respect to

weights yields:

$$\Delta w_i = \eta \cdot (d - \hat{y}) \cdot x_i$$

B(ii) Adaline Implementation (NumPy Only)

Listing 2. Batch gradient descent update (src/adaline.py).

```
y_hat = xb @ self.w
err = y - y_hat
grad = -(xb.T @ err) / n
self.w = self.w - self.lr * grad
mse = float((err**2).mean() / 2.0)
```

B(ii) Loss Surface and Gradient Descent Trajectory

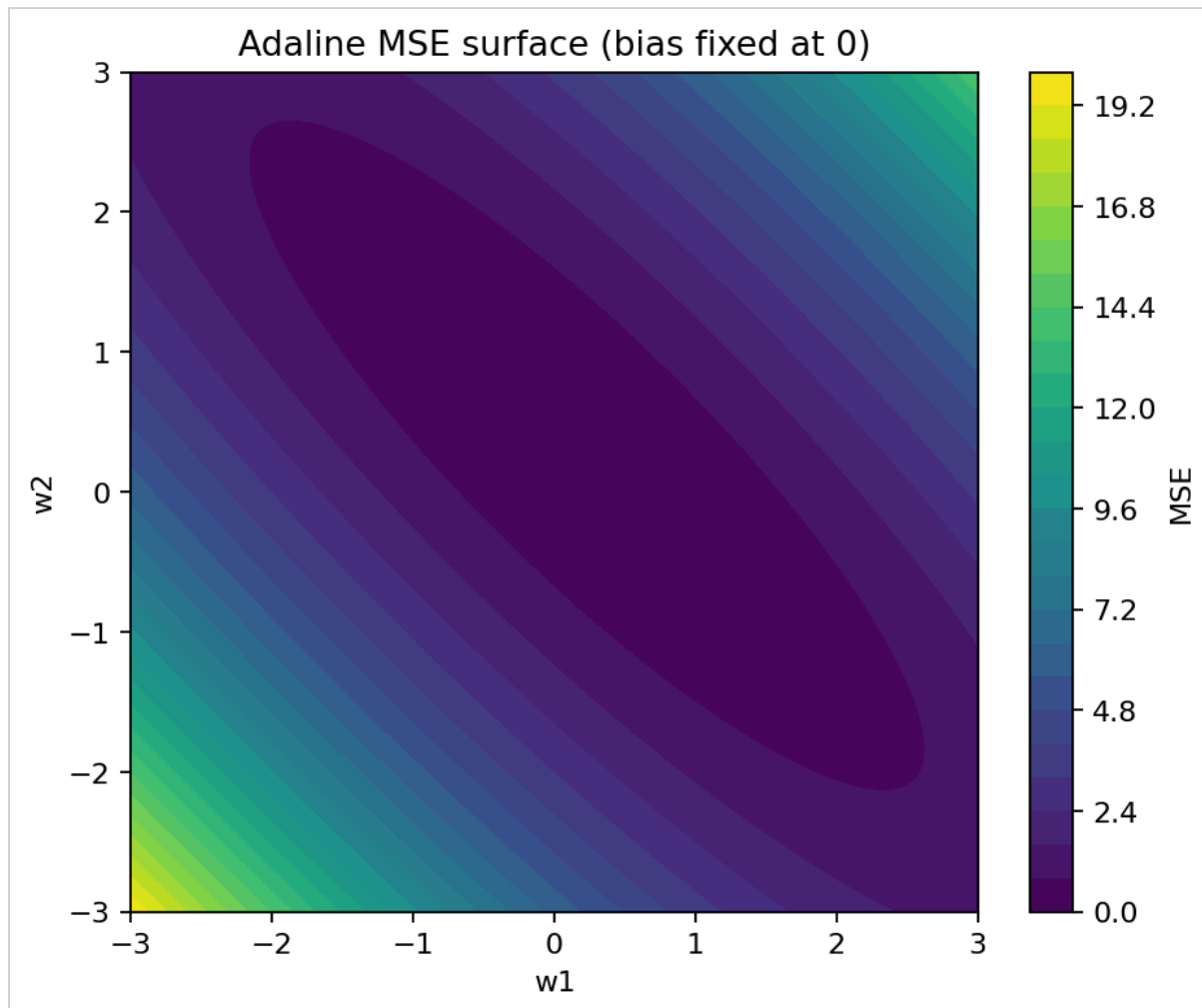


Fig. 11. Adaline MSE loss surface (toy 2D task).

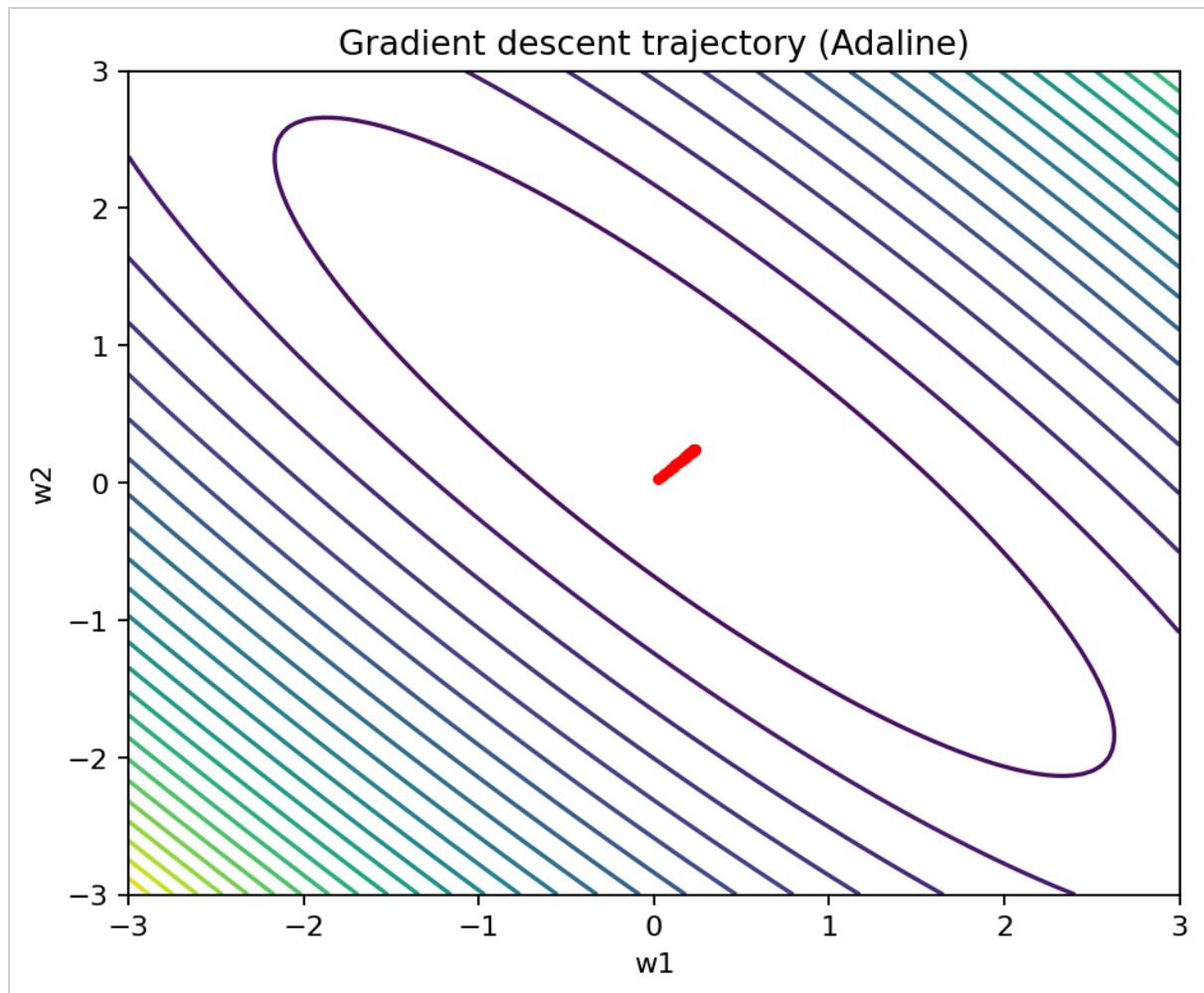


Fig. 12. Gradient descent trajectory in weight space over the MSE surface.

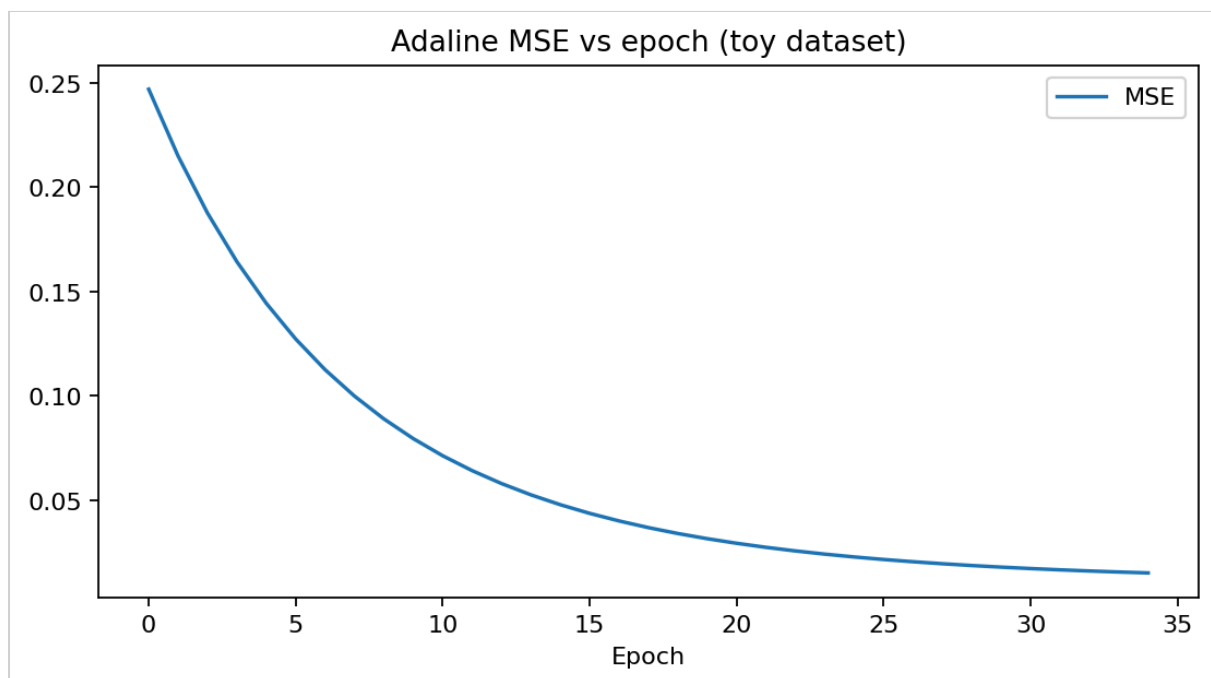


Fig. 13. Adaline MSE vs epoch (toy task).

B(iii) Learning Rate Analysis

We test learning rates $\eta \in \{0.0001, 0.001, 0.01, 0.1\}$. Smaller η is stable but slow; larger η converges faster but can oscillate or diverge depending on curvature. In this experiment, $\eta = 0.1$ converges fastest without divergence on the selected task.

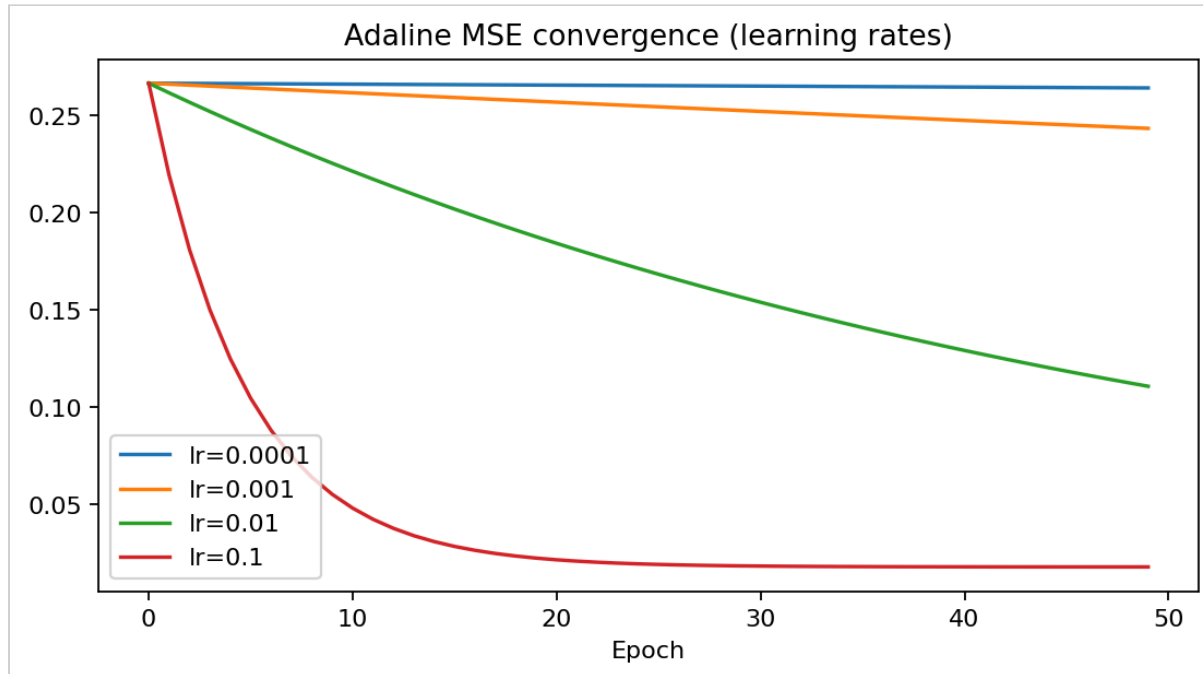


Fig. 14. MSE convergence curves for different learning rates.

IV. Part C: Kohonen Self-Organizing Maps (SOM)

C(i) Architecture and Competitive Learning

$$\begin{aligned} \text{BMU} &= \underset{i}{\operatorname{argmin}} \|x - w_i\| \\ \eta(t) &= \eta_0 \cdot \exp(-t/\lambda), \quad \sigma(t) = \sigma_0 \cdot \exp(-t/\lambda) \\ w_i(t+1) &= w_i(t) + \eta(t) \cdot h(\text{BMU}, i, t) \cdot (x - w_i(t)) \end{aligned}$$

The SOM maps high-dimensional MNIST inputs onto a 2D grid while preserving topological relationships using a Gaussian neighborhood function.

C(ii) SOM Training (NumPy Only)

Listing 3. Local neighborhood update around the BMU (`src/som.py`).

```
bmu_r, bmu_c = self.bmu(xi)
lr_t, sigma_t = self._decay(t, max_t)
rad = int(np.ceil(3.0 * sigma_t))
h = np.exp(-dist2 / (2.0 * (sigma_t**2 + 1e-12))).astype(np.float32)
self.weights[r0:r1+1, c0:c1+1] = w_local + lr_t * h[... , None] * (xi
```

C(iii) Visualization and Evaluation (U-matrix, Prototypes, Hit Maps, QE/TE)

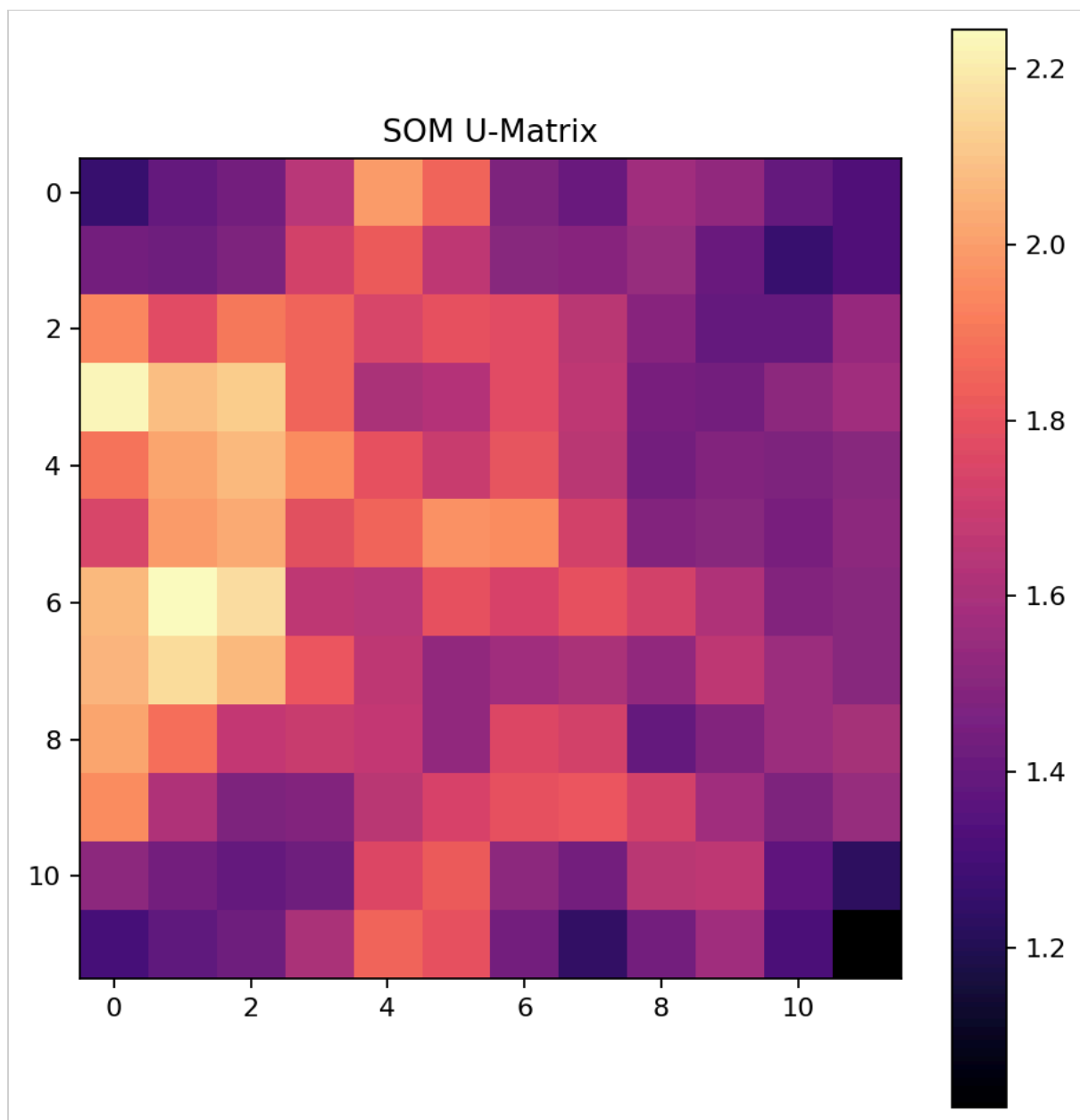


Fig. 15. U-matrix visualization of neighbor distances (cluster boundaries appear as high-distance regions).

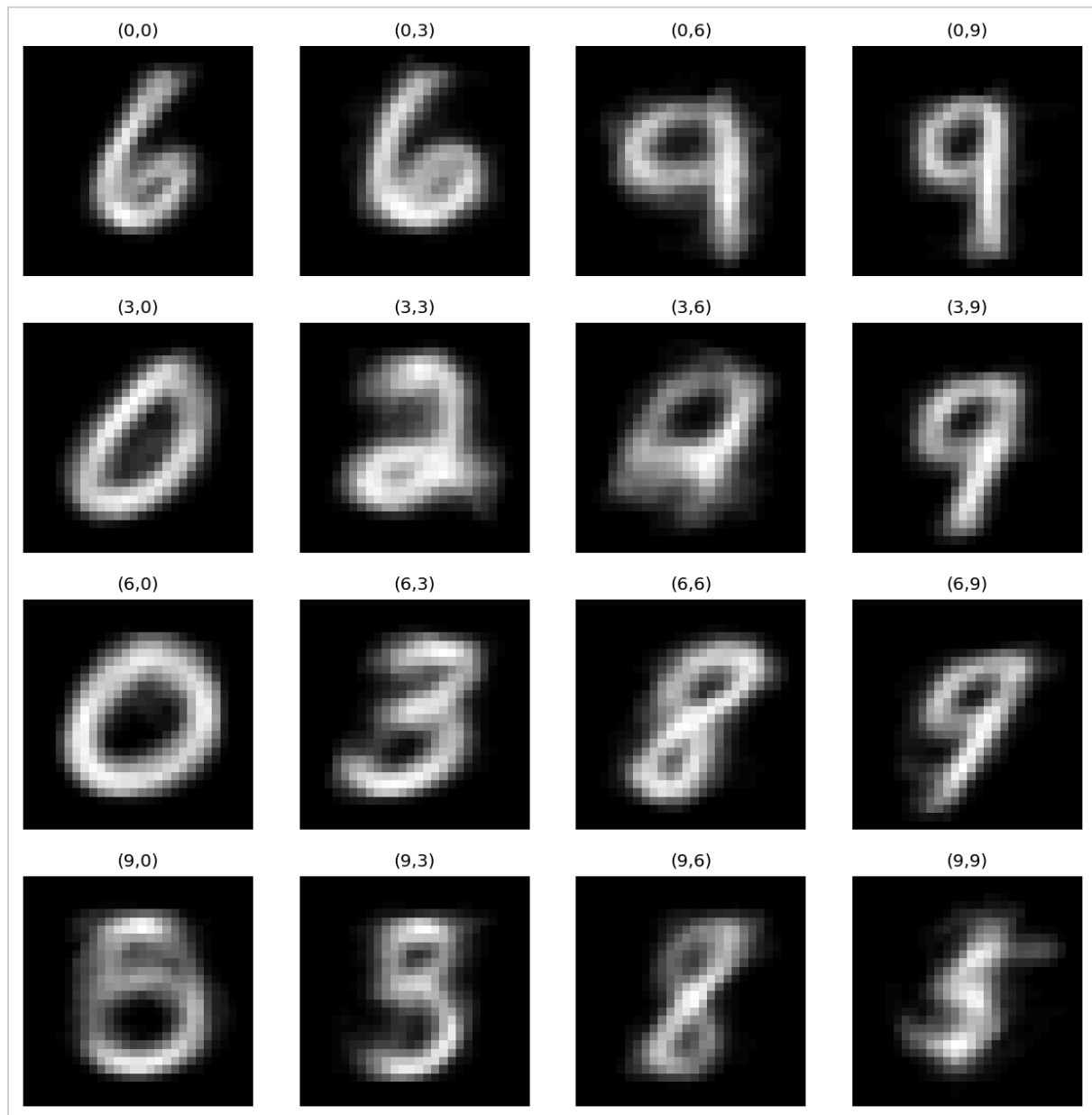


Fig. 16. SOM prototype (weight) maps reshaped to 28×28 images.

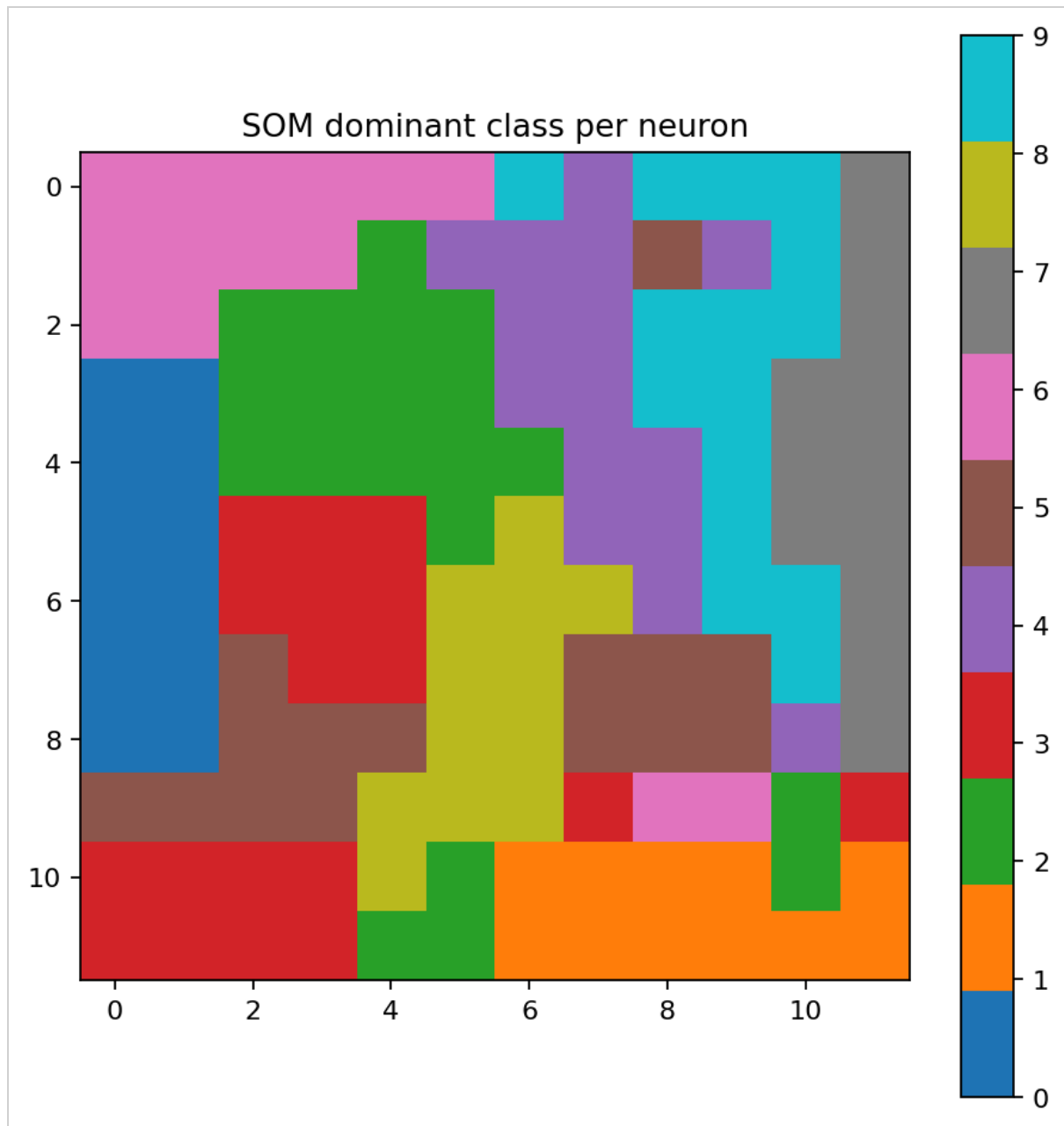


Fig. 17. Class hit map (winner label per neuron from post hoc digit counts).

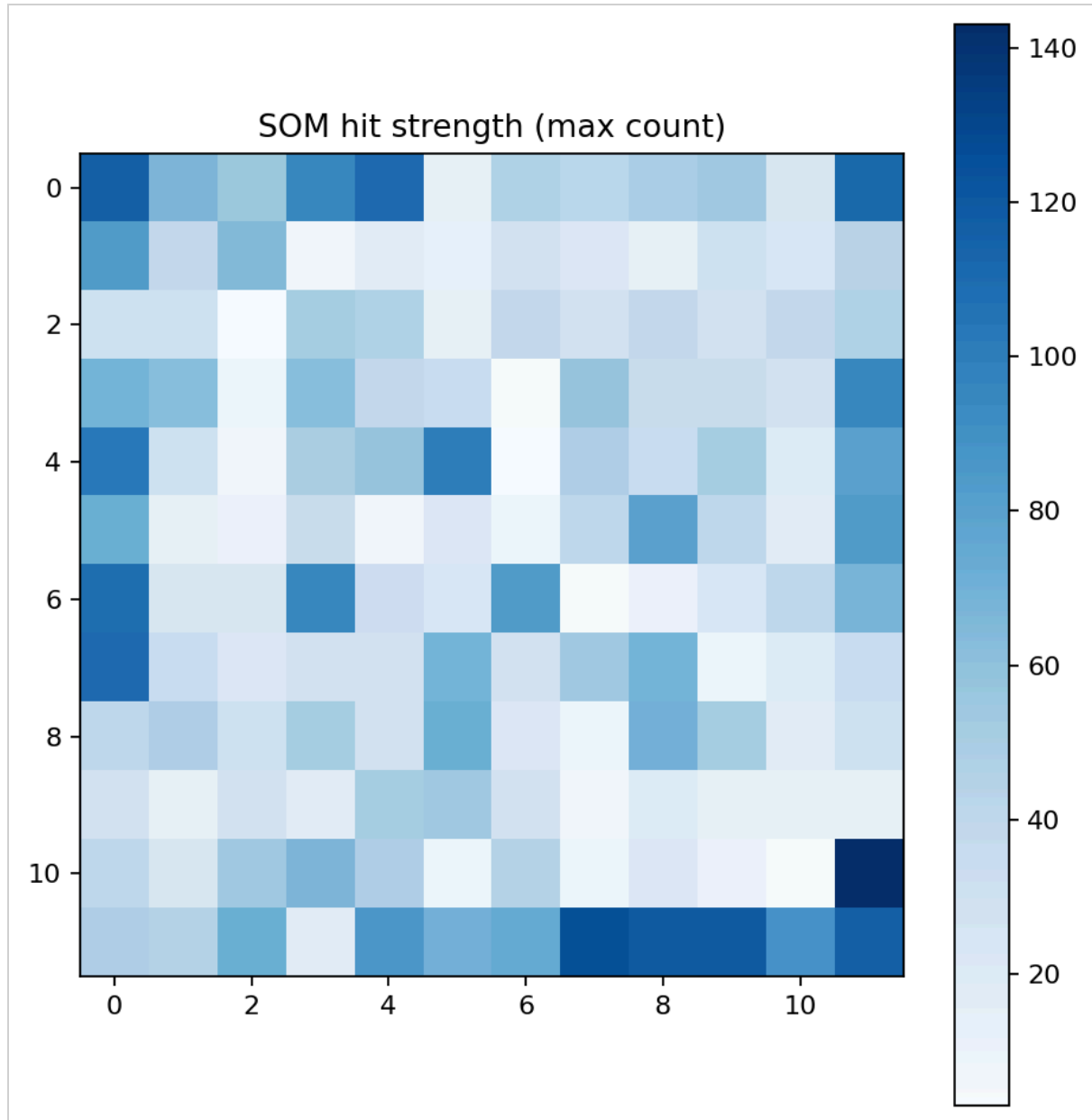


Fig. 18. Hit strength map showing activation density over the SOM grid.

Quality metrics from the notebook run: QE = 5.535064 and TE = 0.10775.

C(iv) Anomaly Detection

We use per-sample quantization error as an anomaly score and choose a threshold from the error distribution (95th percentile of clean errors). At threshold 7.481676, precision = 0.8298755, recall $\approx$ 1.0, and F1 = 0.9070295.

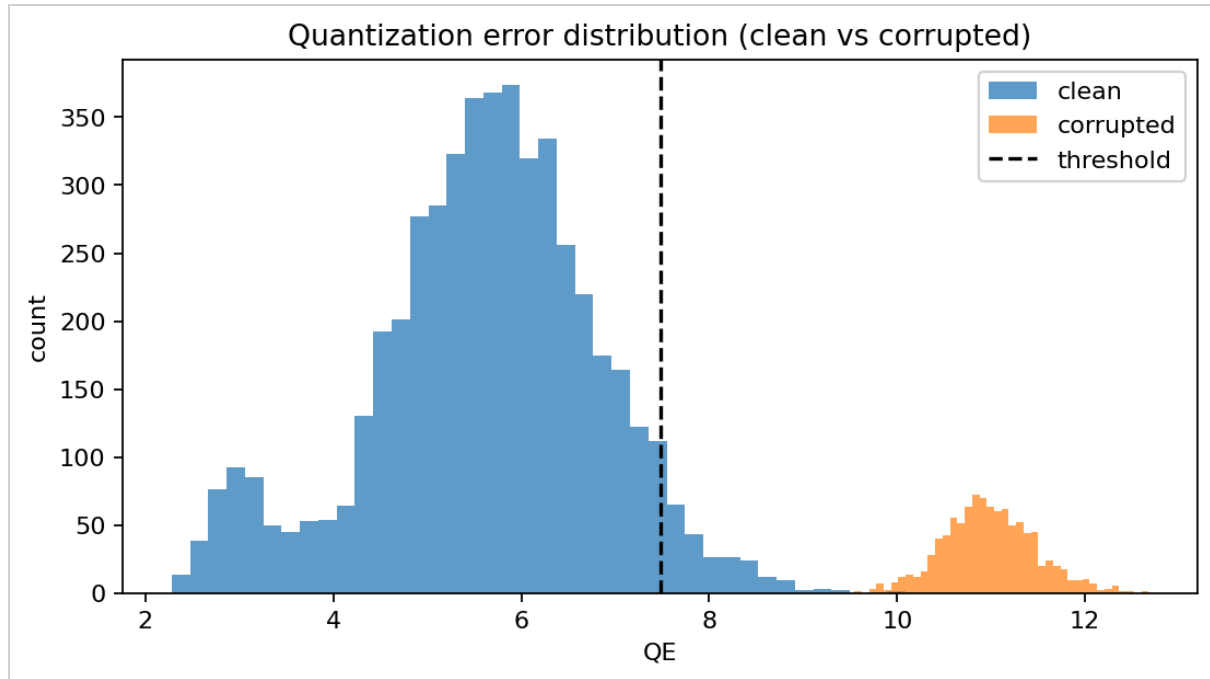


Fig. 19. Quantization error distribution and chosen anomaly threshold.

V. Part D: Backpropagation Networks (MLP)

D(i) Theoretical Derivation (3-layer MLP)

Backpropagation extends the Delta Rule to hidden layers using the chain rule. For output unit k and hidden unit j :

$$\begin{aligned}\delta_k &= (d_k - y_k) \cdot f'(\text{net}_k) \\ \delta_j &= f'(\text{net}_j) \cdot \sum_k \delta_k \cdot w_{kj} \\ \Delta w_{ij} &= \eta \cdot \delta_j \cdot o_i\end{aligned}$$

In practice with softmax + cross-entropy, the gradient at logits simplifies to $(p - y)$.

D(ii) MLP Implementation (NumPy Only): 784→128→64→10

The MLP is implemented from scratch in `src/mlp.py`, including forward pass, cross-entropy loss, and full backpropagation. It uses mini-batch SGD with momentum, optional L2 regularization and dropout, and compares activations.

D(ii) Training Curves and Test Evaluation

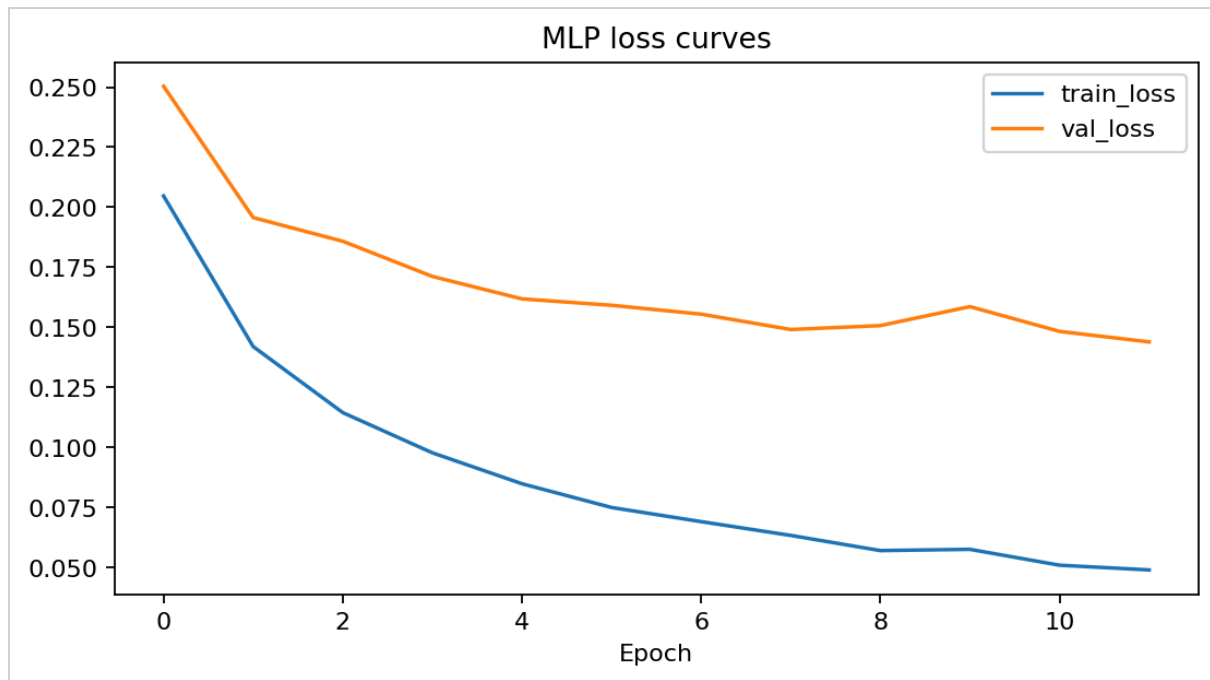


Fig. 20. Training/validation loss curves for the MLP.

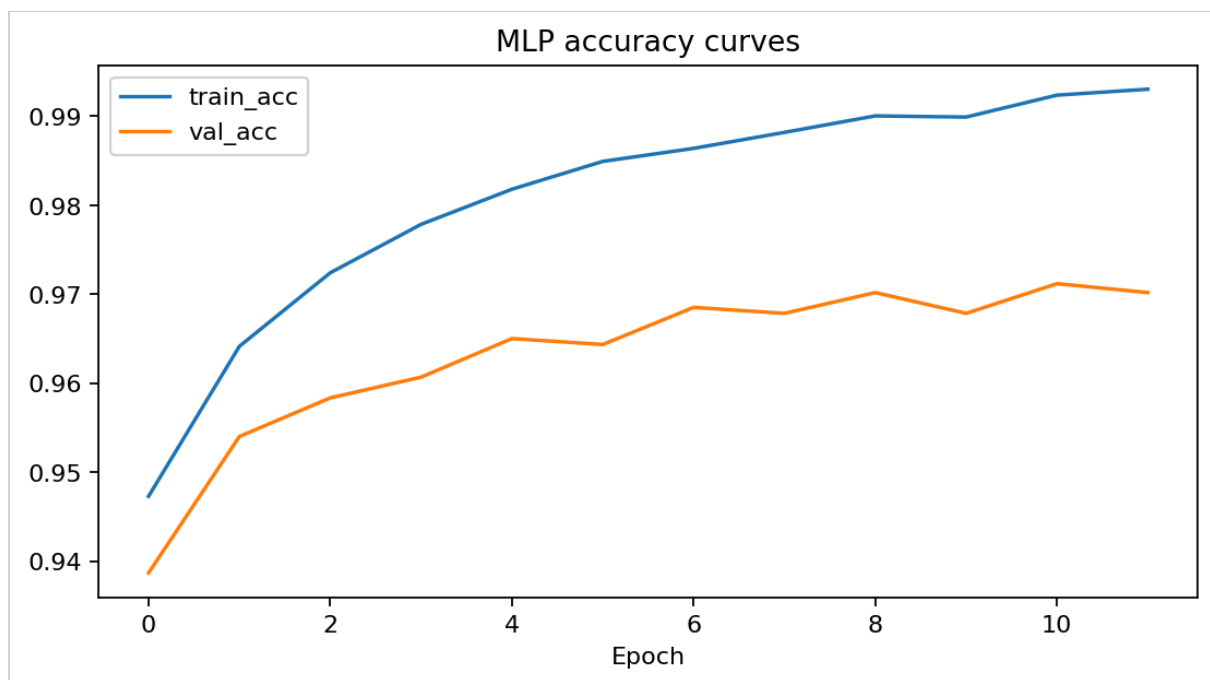


Fig. 21. Training/validation accuracy curves for the MLP.

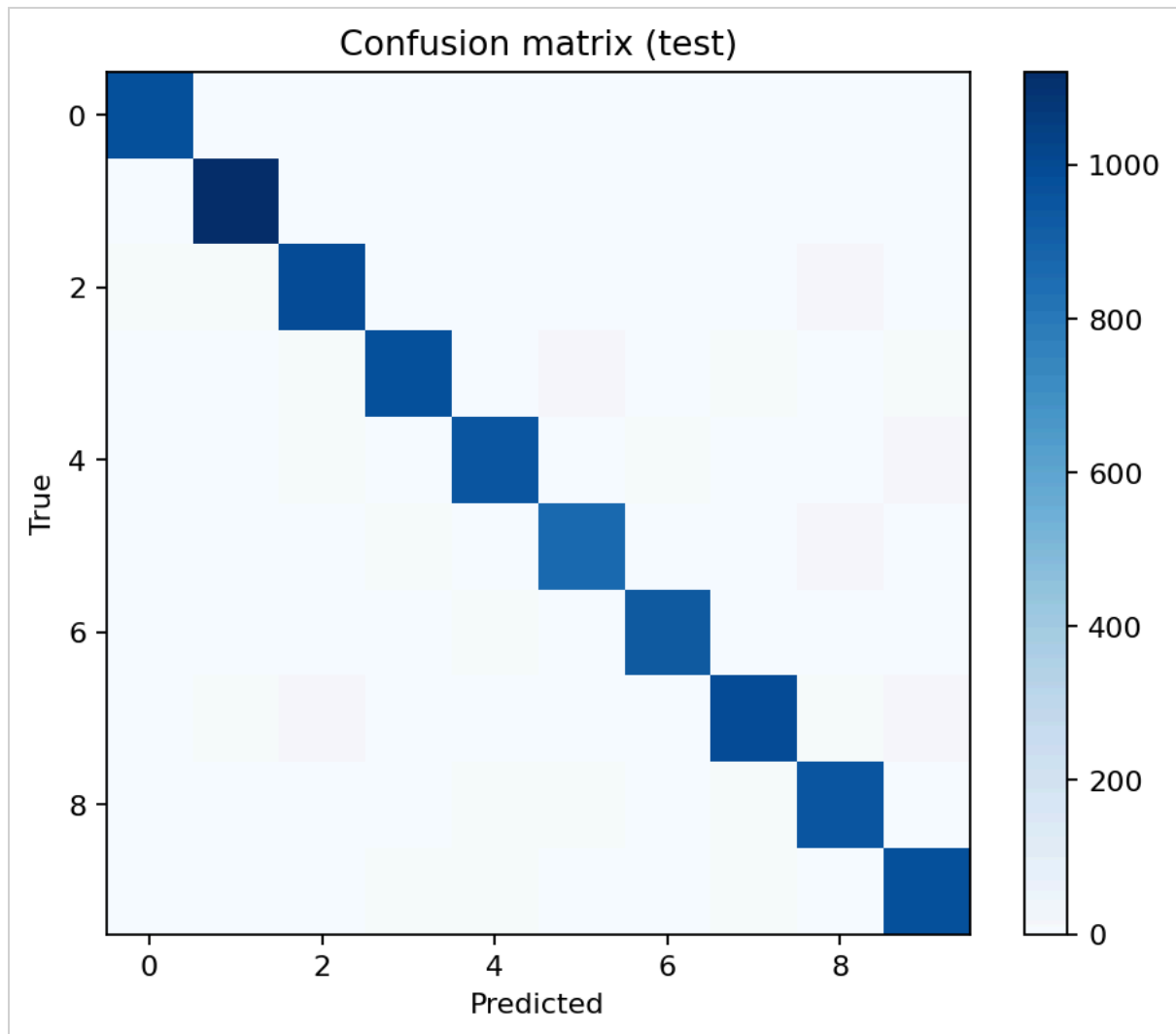


Fig. 22. Confusion matrix on the MNIST test set (final test accuracy: 0.9714).

D(iii) Activation Function Comparison

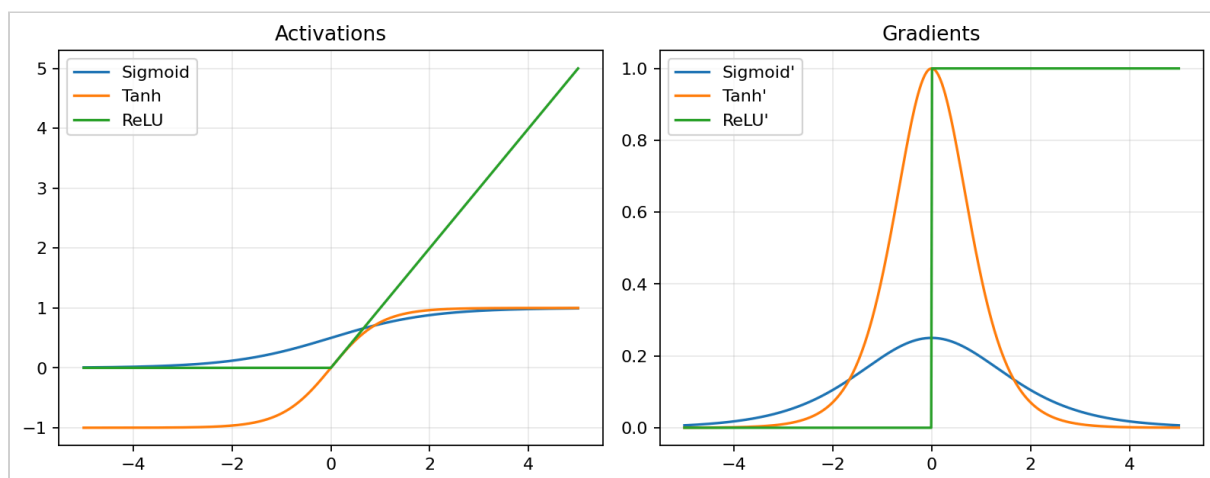


Fig. 23. Sigmoid, tanh, and ReLU activation curves and gradient magnitudes.

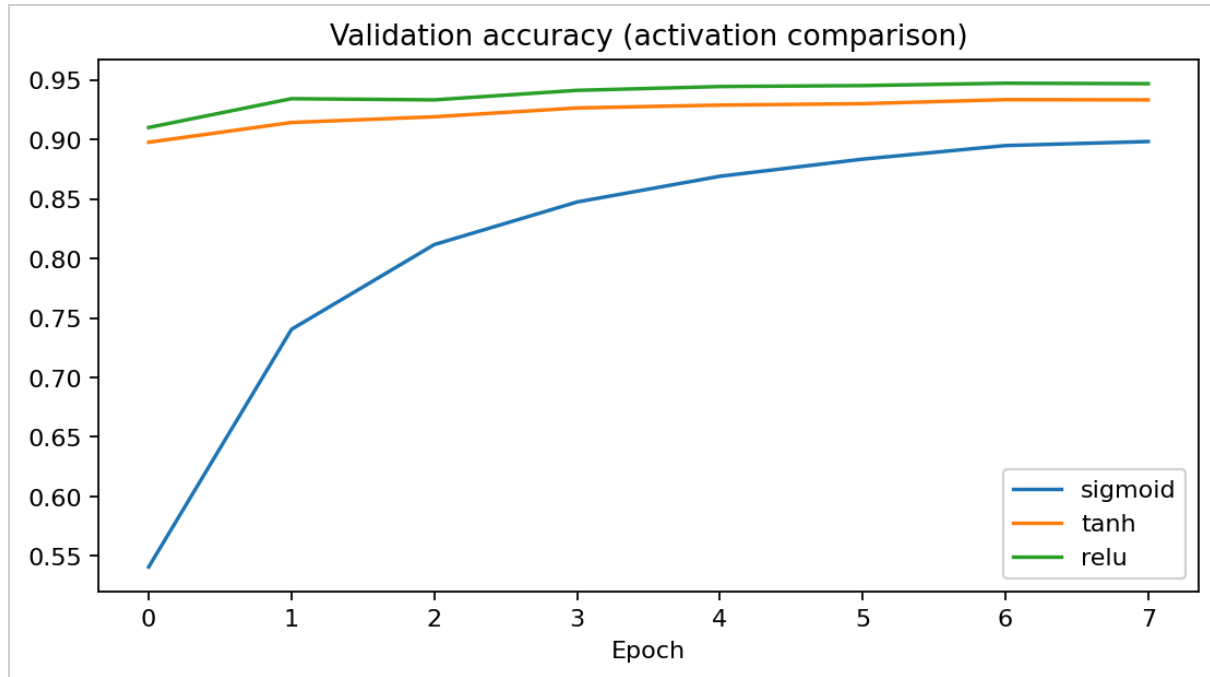


Fig. 24. Validation accuracy across activations (ReLU converges fastest and highest in this run).

D(iv) Regularization (L2 + Dropout)

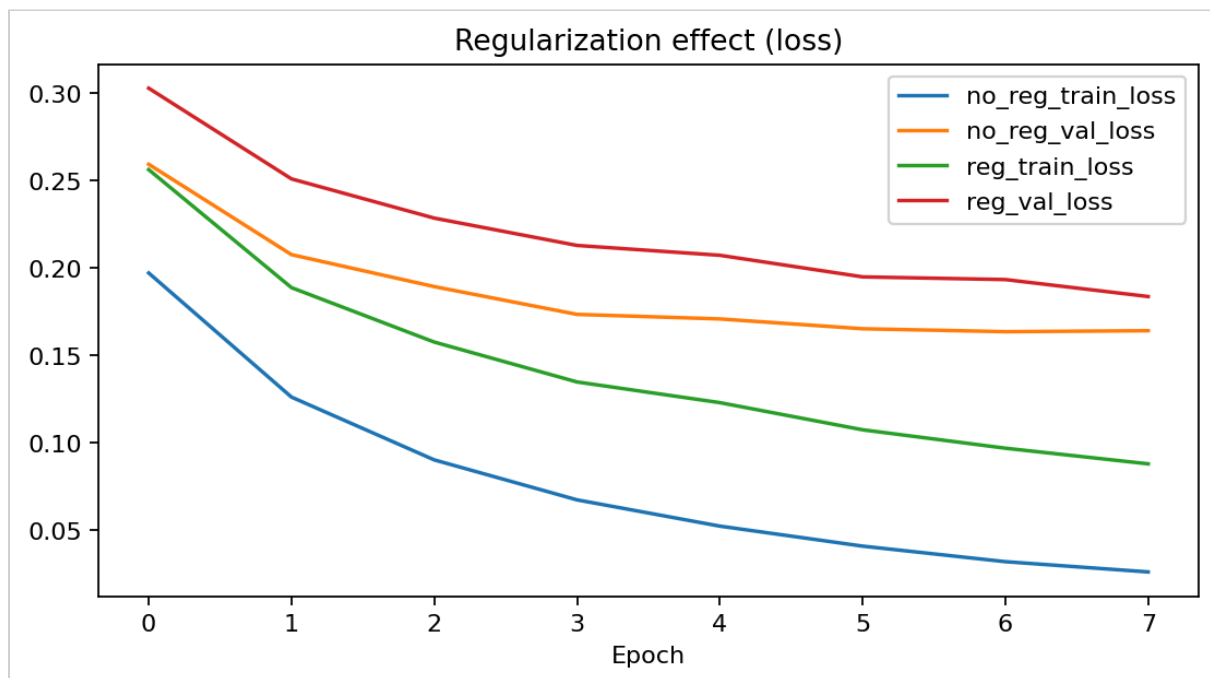


Fig. 25. Effect of regularization on training vs validation loss (reduced overfitting gap).

D(v) SOM Integration

SOM cluster assignments are incorporated as auxiliary features by appending normalized BMU coordinates to the MLP input. In this run, the SOM-augmented model does not improve

final validation accuracy (baseline 0.964833 vs SOM-aug 0.964333), but can slightly affect early-epoch convergence.

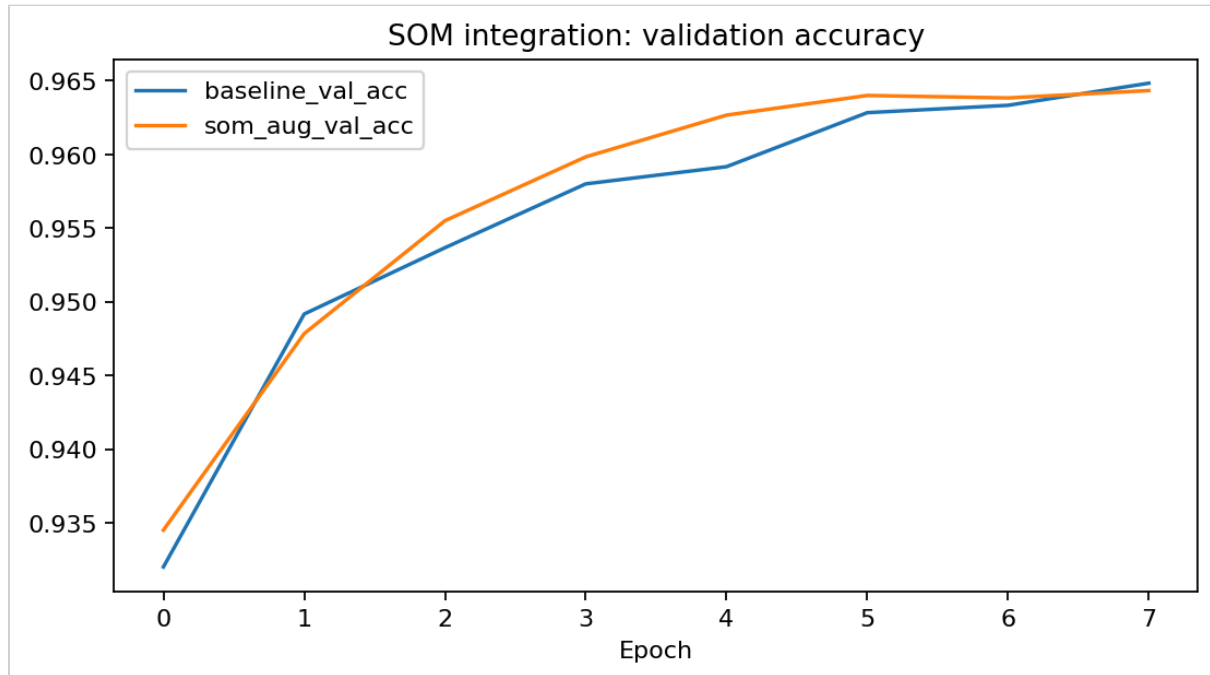


Fig. 26. Validation accuracy comparison: baseline MLP vs SOM-augmented features.

VI. Comparative Analysis

Table II. Paradigm Comparison

Paradigm	Supervision	Objective	Strengths	Limitations
Perceptron	Supervised	Misclassification correction	Simple, fast, interpretable; convergence on separable data	Cannot solve XOR; only linear separability
Adaline	Supervised	MSE minimization (GD)	Differentiable training; clear learning-rate behavior	Still linear decision boundary
SOM	Unsupervised	Topology-preserving quantization	Clustering, visualization, anomaly detection; interpretable prototypes	No direct class optimization; tuning grid/decay required
MLP (Backprop)	Supervised	Cross-entropy minimization	Non-linear representations; high MNIST accuracy	More compute; hyperparameter sensitivity; risk of overfitting

Note: Complexity increases from single-layer linear models to multi-layer backpropagation networks; SOM provides unsupervised structure and interpretability.

VII. Conclusion

The project demonstrates how learning paradigms evolve from linear threshold models to multi-layer gradient-based systems. The Perceptron provides a baseline and illustrates the necessity of linear separability. Adaline introduces differentiable learning and learning-rate sensitivity. SOM adds unsupervised topology-preserving clustering, prototype visualization, and anomaly detection via quantization error. Finally, the NumPy-only MLP trained with backpropagation achieves strong MNIST performance (97.14% test accuracy), validating the representational power of multi-layer networks. Comparative results show clear trade-offs in complexity, interpretability, and performance.

References

1. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.

2. B. Widrow and M. E. Hoff, “Adaptive switching circuits,” in *1960 IRE WESCON Convention Record*, 1960, pp. 96–104.
3. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
4. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
5. T. Kohonen, *Self-Organizing Maps*, 3rd ed. Springer, 2001.
6. M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.

Appendix A: Hyperparameters

Primary hyperparameters from the provided notebook run (notebooks/ccp_ann_project.ipynb):

Table III. Hyperparameters (Notebook Run)

Part	Model	Key settings
A	Perceptron (PCA-2D, 0 vs 1)	Binary MNIST subset; boundary evolution saved across epochs; update rule $w \leftarrow w + \eta(d - y)x$.
B	Adaline (GD)	Batch GD on MSE; $\eta \in \{0.0001, 0.001, 0.01, 0.1\}$; 50 epochs for convergence comparison plot.
C	SOM	Grid 12×12 ; $lr_0 = 0.5$; $\sigma_0 = 6.0$; epochs = 6; exponential decay; local neighborhood updates ($\approx 3\sigma$ window).
D	MLP	$784 \rightarrow 128 \rightarrow 64 \rightarrow 10$; activation = ReLU; $lr = 0.01$; batch size = 128; momentum = 0.9; $L_2 = 1e-4$; dropout = 0.1 (baseline run).
